

第 8 週：期中考

三道題，空白的設計檔給你填，測試平台會自動判分。

本週指令

雙擊 `env.bat` 開好環境（提示字元要有 `[OSS CAD Suite]`），然後：

```
cd week08_midterm
```

每個範例都是這三條（外加一條檢查），把 `01_seg7` 換成你要跑的名字：

```
iverilog -o sim.out 01_seg7.v 01_seg7_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_seg7.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_seg7`
- `02_updown`
- `03_pattern_fsm`

下面每個範例的段落，都直接附了它自己的四行指令。

怎麼考

```
cd week08_midterm
iverilog -o sim.out 01_seg7.v 01_seg7_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

打開 `01_seg7.v`，找到 `★★★` 在這裡寫你的答案 `★★★`，把它補完，存檔後把上面三行重跑一次（按 `↑` 上方向鍵 叫回指令），測試會告訴你得幾分、哪裡錯。

考試看終端機的分數就好，不一定要開波形。想看波形再加一行 `gtkwave 01_seg7.gtkw`。

[!WARNING] 不要改 `_tb.v` 檔。那是考卷，改了就沒意義了。解答放在 `_answers/`，先自己寫過再看。

前 7 週重點複習

觀念地圖

```
第1週  module / 腳位 / assign / 實例化
      ↓
第2週  數字寬度 / 切片 / 串接 / 五類運算子 / x 和 z
      ↓
第3週  always @(*) vs always @(posedge clk)  ← 最重要
      = vs <=                                ← 最重要
      latch 陷阱
      ↓
第4週  function / parameter / generate
      ↓
第5週  自我檢查測試平台 / $strobe / 亂數測試
      ↓
第6週  三段式狀態機 / Moore vs Mealy
      ↓
第7週  閘延遲 / 毛刺 / 亞穩態 / 閘級模擬
```

必背的三條規則

規則	內容
1	<code>always @(posedge clk)</code> 裡用 <code><=</code> ; <code>always @(*)</code> 裡用 <code>=</code>
2	組合邏輯的每條路徑都要給值 (<code>else</code> 和 <code>default</code> 一定要寫) , 否則產生 latch
3	在 <code>always</code> 裡被賦值的訊號宣告成 <code>reg</code> , 用 <code>assign</code> 的宣告成 <code>wire</code>

常見錯誤清單

症狀	通常的原因
波形一片紅色 x	reg 沒給初值 / reset 沒接好 / 忘了實例化的某個腳
波形出現中線 z	wire 沒人驅動 / 三態沒開 enable
值差一拍	<code>\$display</code> 印到舊值 (用 <code>\$strobe</code>) 或 <code>=</code> / <code><=</code> 用錯
訊號卡住不動	latch (漏了 <code>else</code> 或 <code>default</code>)
計算結果變小	位元寬度不夠被截斷 (<code>4'd9 + 4'd8 = 1</code> 不是 17)
三行程式碼只做出一顆 FF	在時脈區塊用了 <code>=</code>
綜合有 warning 說 latch	同上, 補預設值

三道題

第 1 題：01_seg7 — BCD 七段顯示解碼器

難度 ★☆☆ 考第 2、3 週

輸入 4 位元 BCD，輸出 7 段字型。10~15 全滅。

```
iverilog -o sim.out 01_seg7.v 01_seg7_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

- 滿分 16 分 (0~15 每個值 1 分)
- 通過後測試會用 ASCII 把「8」畫給你看

► 提示

第 2 題：02_updown — 可預載的上下數計數器

難度 ★★☆☆ 考第 3、4 週

優先順序：rst > load > en。還要輸出 at_max / at_min。

```
iverilog -o sim.out 02_updown.v 02_updown_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

- 滿分 256 分 (含 200 拍亂數操作)
- 參考模型會跟著你的設計同步跑，任何一拍對不上就扣分

► 提示

第 3 題：03_pattern_fsm — 密碼鎖狀態機

難度 ★★★ 考第 6 週 + 綜合

密碼 1 → 2 → 3 → 0，按對開鎖一拍，按錯重來。

```
iverilog -o sim.out 03_pattern_fsm.v 03_pattern_fsm_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

- 滿分 17 分，而且開鎖次數必須剛好 3 次
- ⚠ 陷阱：按錯的那個鍵如果剛好是密碼第一碼，要算成新的開始

► 提示

評分標準

分數	意思
三題都滿分	前 7 週紮實，直接進第 9 週
第 1、2 題滿分	基礎沒問題，第 3 題再想想，可以進第 9 週
第 2 題不到一半	回去重讀第 3 週， <code>= / <=</code> 和優先順序沒通
第 1 題不滿分	回去重讀第 2、3 週的 <code>case</code> 和 <code>default</code>

寫完之後

三題都通過了，去 `_answers/` 對照一下：別人的寫法跟你哪裡不一樣？為什麼？這一步的學習效果常常比自己寫還大。

想對照參考解答

解答在 `_answers\` 裡。先自己寫過再看。想直接跑解答版驗證：

```
iverilog -o sim.out _answers\01_seg7.v 01_seg7_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

(就是把設計檔的路徑換成 `_answers\` 裡的那份，測試檔不變)

附註：關於 `sim` 這個指令

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔 (`sim.bat`)，不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。